

DOKUMENTACJA PROJEKTOWA

LARPHub

System wspierający organizację komnatowych wydarzeń LARP

Autor: Dawid Bałdyga

Firma: Politechnika Warszawska

Przedmiot: Inżynieria oprogramowania

Status: roboczy

Wersja: 1.0

Data modyfikacji: 10.06.2026

Abstrakt: Dokument zawiera specyfikację projektową systemu LARPHub wspierającego organizację komnatowych wydarzeń LARP. Do zapisania wymagań funkcjonalnych, słownika dziedziny, projektu struktury systemu oraz jego dynamiki wykorzystano notację języka UML oraz notację scenariuszy RSL-bis.

Historia zmian:

Wersja	Data	Kto	Opis
0.1	01.06.2026	Dawid Bałdyga	Utworzenie dokumentu. Zapoznanie się z tematem projektu i wymaganiami.
0.2	03.06.2026	Dawid Bałdyga	Wykonanie rejestru produktowego oraz rejestru pierwszego sprintu.
0.3	05.06.2026	Dawid Bałdyga	Przygotowanie diagramów przypadków użycia. Opracowanie scenariuszy w notacji RSL-bis oraz diagramu czynności.
0.4	07.06.2026	Dawid Bałdyga	Wykonanie diagramów klas: aktorzy oraz słownik dziedziny problemu (enumeracje, kompozycja, agregacja).
0.5	08.06.2026	Dawid Bałdyga	Utworzenie diagramów komponentów wraz z opisem architektury warstwowej.
0.6	09.06.2026	Dawid Bałdyga	Przygotowanie diagramów sekwencji oraz opisu dynamiki systemu.
0.7	10.06.2026	Dawid Bałdyga	Utworzenie diagramu klas realizacji interfejsu. Wygenerowanie kodu w narzędziu CASE.
1.0	10.06.2026	Dawid Bałdyga	Sfinalizowanie dokumentacji. Sprawdzenie treści i spójności diagramów.

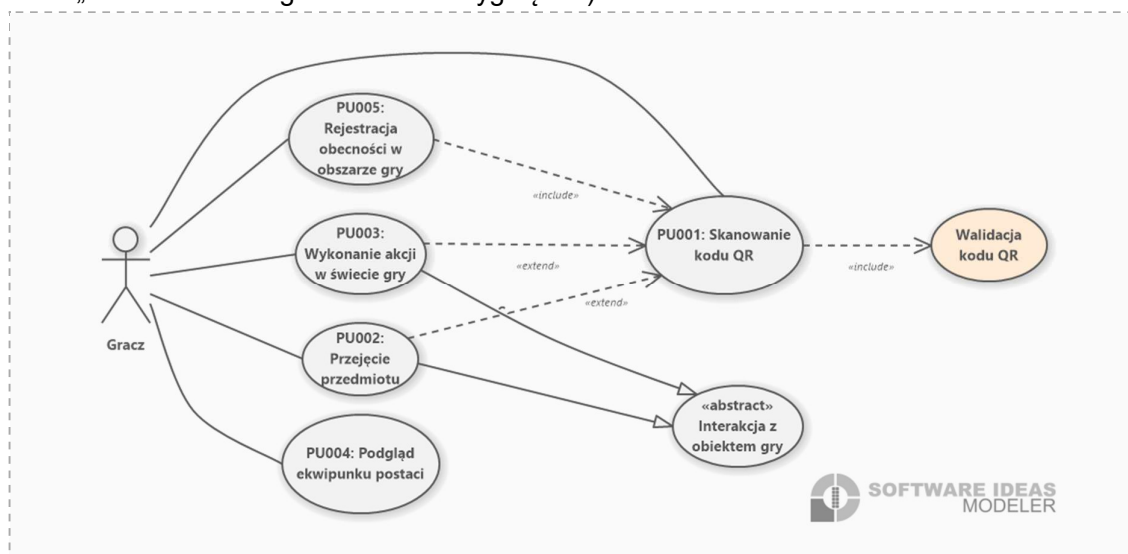
1. Specyfikacja wymagań

1.1 Model przypadków użycia

System LARPHub obejmuje wiele obszarów funkcjonalnych. Zgodnie z zaleceniem ograniczenia zakresu do funkcji najważniejszych i najbardziej złożonych technicznie, specyfikację skupiono na rdzeniu rozgrywki - interakcji gracza ze światem gry oraz na zarządzaniu przebiegiem gry i rozstrzyganiu akcji. Świadomie pominięto funkcjonalność generyczną (logowanie, rejestracja konta).

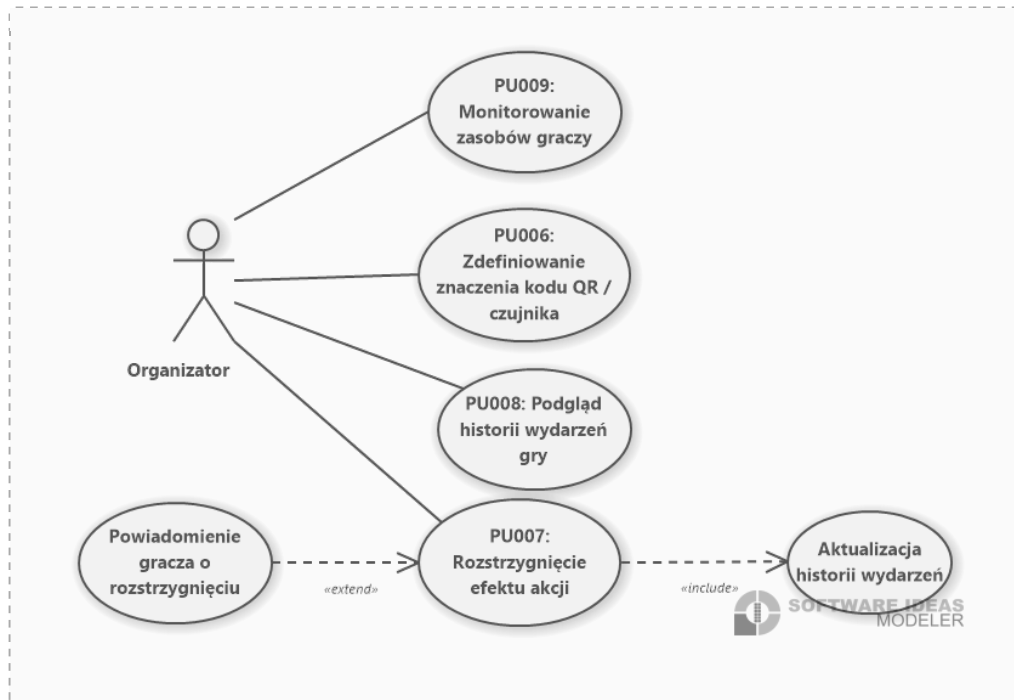
Aktorami systemu są: Gracz (uczestnik wydarzenia korzystający z aplikacji mobilnej) oraz Organizator (Mistrz Gry korzystający z aplikacji webowej). Obaj są wyspecjalizowaniem ogólnego aktora Użytkownik (Rysunek 4). Wyróżniono dwa obszary funkcjonalne:

- Interakcja gracza ze światem gry (aktor Gracz): PU001 Skanowanie kodu QR, PU002 Przejęcie przedmiotu, PU003 Wykonanie akcji w świecie gry, PU004 Podgląd ekwipunku postaci, PU005 Rejestracja obecności w obszarze gry.
- Przebieg gry i rozstrzyganie akcji (aktor Organizator): PU006 Zdefiniowanie znaczenia kodu QR / czujnika, PU007 Rozstrzygnięcie efektu akcji, PU008 Podgląd historii wydarzeń gry, PU009 Monitorowanie zasobów graczy. W modelu przypadków użycia zastosowano generalizację przypadków (abstrakcyjny przypadek „Interakcja z obiektem gry” uogólnia PU002 i PU003), zależności «include»/«extend» oraz przypadki pomocnicze („Walidacja kodu QR”, „Aktualizacja historii wydarzeń”, „Powiadomienie gracza o rozstrzygnięciu”).



Rysunek 1: Diagram przypadków użycia - obszar „Interakcja gracza ze światem gry”

Aktor Gracz oraz PU001–PU005; PU002 i PU003 rozszerzają («extend») PU001, a PU005 zawiera («include») PU001. Dodatkowo PU001 zawiera («include») pomocniczy przypadek „Walidacja kodu QR”, a PU002 i PU003 są uogólnione (generalizacja) do abstrakcyjnego przypadku „Interakcja z obiektem gry”.



Rysunek 2: Diagram przypadków użycia - obszar „Przebieg gry i rozstrzyganie akcji”

Aktor Organizator oraz PU006–PU009. Efekty akcji zlecane w PU003 są rozstrzygane w PU007. PU007 zawiera («include») pomocniczy przypadek „Aktualizacja historii wydarzeń”, a opcjonalne „Powiadomienie gracza o rozstrzygnięciu” rozszerza («extend») PU007.

1.2 Scenariusze przypadków użycia dla sprintu 1

Scenariusze zapisano w notacji RSL-bis (zgodnie z plikiem „Zalecana notacja scenariuszy”). Słowa kluczowe notacji pozostawiono w języku angielskim (<select>, <enter>, <invoke>, <show>, <read>, <check>, <update>, <execute>), a nazwy elementów dziedziny, widoków i wyzwalaczy podano po polsku. Specyfikacja zawiera wymagany przypadek „Start”. Dla sprintu 1 wybrano przypadki PU001, PU002, PU003, PU004 oraz PU007.

Start

```

Use case Start Main scenario
00: Gracz <select> aplikacja gracza
01: System <show> menu główne
02: Gracz <invoke> Skanowanie kodu QR, Podgląd ekwipunku postaci
-> rejoin 01
  
```

PU001: Skanowanie kodu QR

```

Use case Skanowanie kodu QR Main scenario
00: Gracz <select> skanuj kod QR
01: System <show> widok skanera
02: Gracz <enter> odczyt kodu
03: Gracz <select> potwierdź odczyt
04: System <check> kod QR
[kod QR ? PRAWDŹLIWY]
05: System <read> obiekt gry
06: System <show> karta obiektu gry
07: Gracz <select> zamknij
-> end ! OK
  
```

Scenario

```

04: -"-
[kod QR ? NIEPRAWDŹLIWY]
  
```

```
A1: System <show> komunikat o błędnym kodzie
A2: Gracz <select> zamknij
-> end ! BŁĄD
```

Scenario

```
06: -"-
B1: Gracz <invoke> Przejęcie przedmiotu, Wykonanie akcji w świecie gry
-> rejoin 05
```

PU002: Przejęcie przedmiotu

Use case Przejęcie przedmiotu Main scenario

```
{przedmiot}
00: Gracz <select> przejmij przedmiot
01: System <show> potwierdzenie przejęcia
02: Gracz <select> potwierdź
03: System <check> ekwipunek postaci
[ekwipunek postaci ? JEST_MIEJSCE]
04: System <update> ekwipunek postaci
05: System <update> zasób przedmiotu
06: System <show> komunikat o sukcesie
07: Gracz <select> ok
-> end ! OK
```

Scenario

```
03: -"-
[ekwipunek postaci ? BRAK_MIEJSCA]
A1: System <show> komunikat o braku miejsca
A2: Gracz <select> ok
-> end ! ODRZUCONO
```

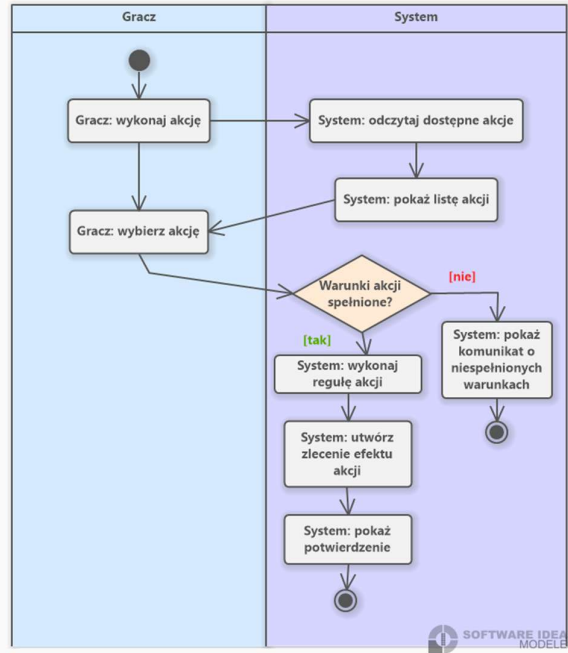
PU003: Wykonanie akcji w świecie gry

Use case Wykonanie akcji w świecie gry Main scenario

```
{punkt akcji}
00: Gracz <select> wykonaj akcję
01: System <read> dostępne akcje
02: System <show> lista akcji
03: Gracz <select> wybierz akcję
04: System <check> warunki akcji
[warunki akcji ? SPEŁNIONE]
05: System <execute> reguła rozstrzygnięcia akcji
06: System <update> zlecenie efektu akcji
07: System <show> potwierdzenie zlecenia akcji
08: Gracz <select> ok
-> end ! OK
```

Scenario

```
04: -"-
[warunki akcji ? NIESPEŁNIONE]
A1: System <show> komunikat o niespełnionych warunkach
A2: Gracz <select> ok
-> end ! ODRZUCONO
```



Rysunek 3: Diagram czynności (aktywności) dla PU003 - Wykonanie akcji w świecie gry (z torami)

Scenariusz PU003 przedstawiony jako diagram czynności (węzeł decyzyjny dla warunków akcji). Zastosowano tory (partycje) „Gracz” oraz „System”, rozdzielające odpowiedzialności aktora i systemu.

PU004: Podgląd ekwipunku postaci

Use case Podgląd ekwipunku postaci Main scenario

```
00: Gracz <select> pokaż ekwipunek
01: System <read> postać
02: System <read> ekwipunek postaci
03: System <show> karta postaci
04: Gracz <select> zamknij
-> end ! OK
```

PU007: Rozstrzygnięcie efektu akcji

Use case Rozstrzygnięcie efektu akcji Main scenario

```
00: Organizator <select> rozstrzygnij efekty
01: System <read> lista zleconych efektów
02: System <show> lista zleconych efektów
03: Organizator <select> wybierz efekt
04: System <read> efekt akcji
05: System <show> karta efektu akcji
06: Organizator <enter> decyzja o efekcie
07: Organizator <select> zatwierdź
08: System <check> decyzja o efekcie
[decyzja o efekcie ? PRAWIDŁOWA]
09: System <update> stan zasobów gry
10: System <update> historia wydarzeń
11: System <show> komunikat o rozstrzygnięciu
12: Organizator <select> ok
-> end ! OK
```

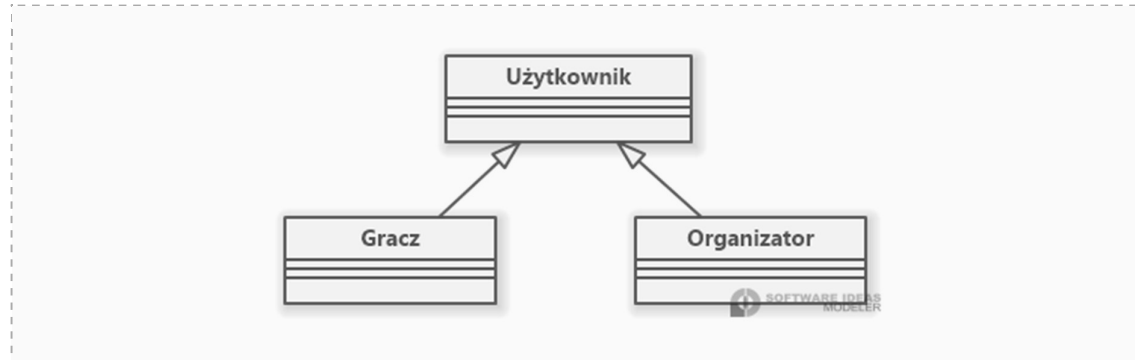
Scenario

```
08: "-"
[decyzja o efekcie ? NIEPRAWIDŁOWA]
A1: System <show> komunikat o błędnej decyzji
A2: Organizator <select> ok
```

-> rejoin 05

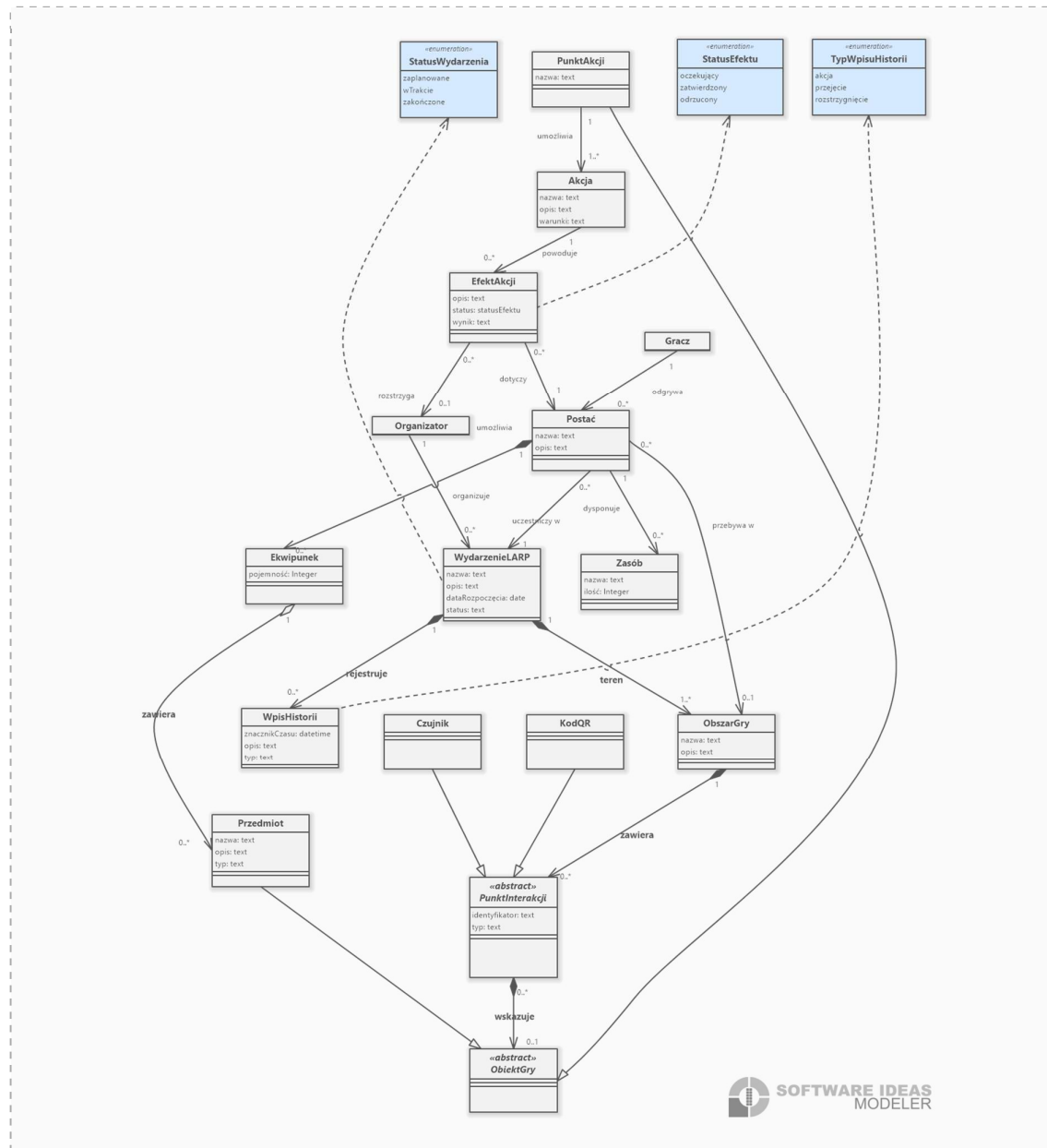
1.3 Słownik dziedziny problemu

Słownik dziedziny problemu przedstawiono w postaci diagramów klas. Rysunek 4 porządkuje aktorów systemu, a Rysunek 5 - pojęcia dziedziny wraz z atrybutami, krotnościami i rolami końców relacji. Pojęcia są spójne z nazwami przypadków użycia i z elementami dziedziny użytymi w scenariuszach (m.in. kod QR, obiekt gry, przedmiot, ekwipunek, zasób, akcja, efekt akcji, historia wydarzeń). Słownik wzbogacono o enumeracje (StatusWydarzenia, StatusEfektu, TypWpisuHistorii) typujące wybrane atrybuty oraz o relacje kompozycji i agregacji.



Rysunek 4: Diagram klas - aktorzy systemowi

Generalizacja: Użytkownik ← Gracz, Organizator.



Rysunek 5: Diagram klas - słownik dziedziny problemu (z enumeracjami i kompozycją)

Klasy dziedziny, atrybuty, krotności i role relacji. Uwzględniono trzy enumeracje (*StatusWydarzenia*, *StatusEfektu*, *TypWpisuHistorii*) oraz kompozycje (*Wydarzenie–Obszar gry*, *Obszar gry–Punkt interakcji*, *Postać–Ekwipunek*) i agregację (*Ekwipunek–Przedmiot*).

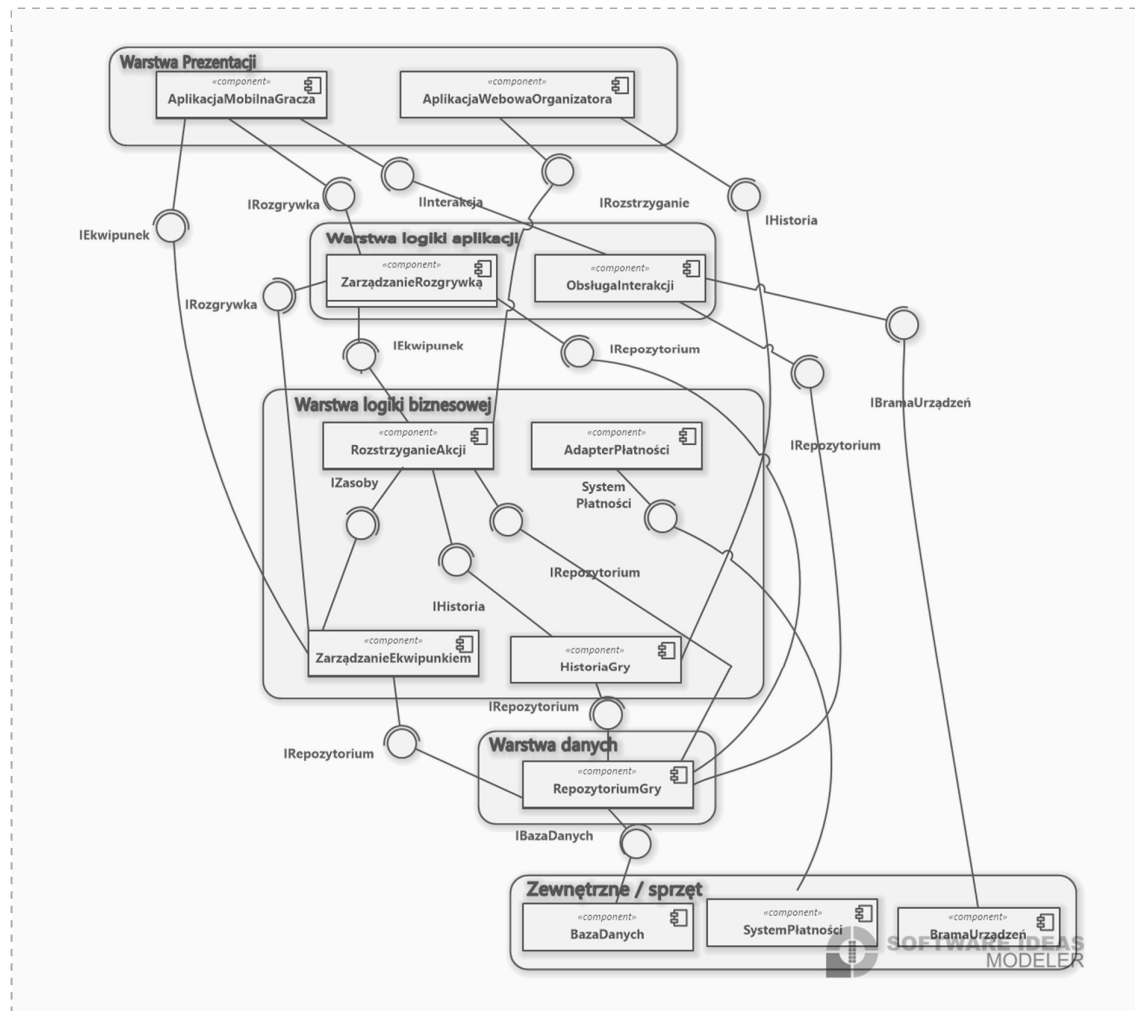
Najważniejsze pojęcia: Wydarzenie LARP składa się z Obszarów gry, które zawierają Punkty interakcji (Kod QR, Czujnik) wskazujące Obiekty gry (Przedmiot lub Punkt akcji). Gracz odgrywa Postać uczestnicząc w Wydarzeniu; Postać posiada Ekwipunek (zawierający Przedmioty) oraz dysponuje Zasobami. Punkt akcji umożliwia Akcje, które powodują Efekty akcji dotyczące Postaci i rozstrzygane przez Organizatora. Przebieg gry jest utrwalany jako Wpisy historii powiązane z Wydarzeniem. Statusy i typy wyrażono jako enumeracje, a złożenia całość–część (*Wydarzenie–Obszar gry*, *Obszar gry–Punkt interakcji*, *Postać–Ekwipunek*, *Ekwipunek–Przedmiot*) jako kompozycje/agregacje.

2. Projekt systemu

2.1 Model komponentów systemu

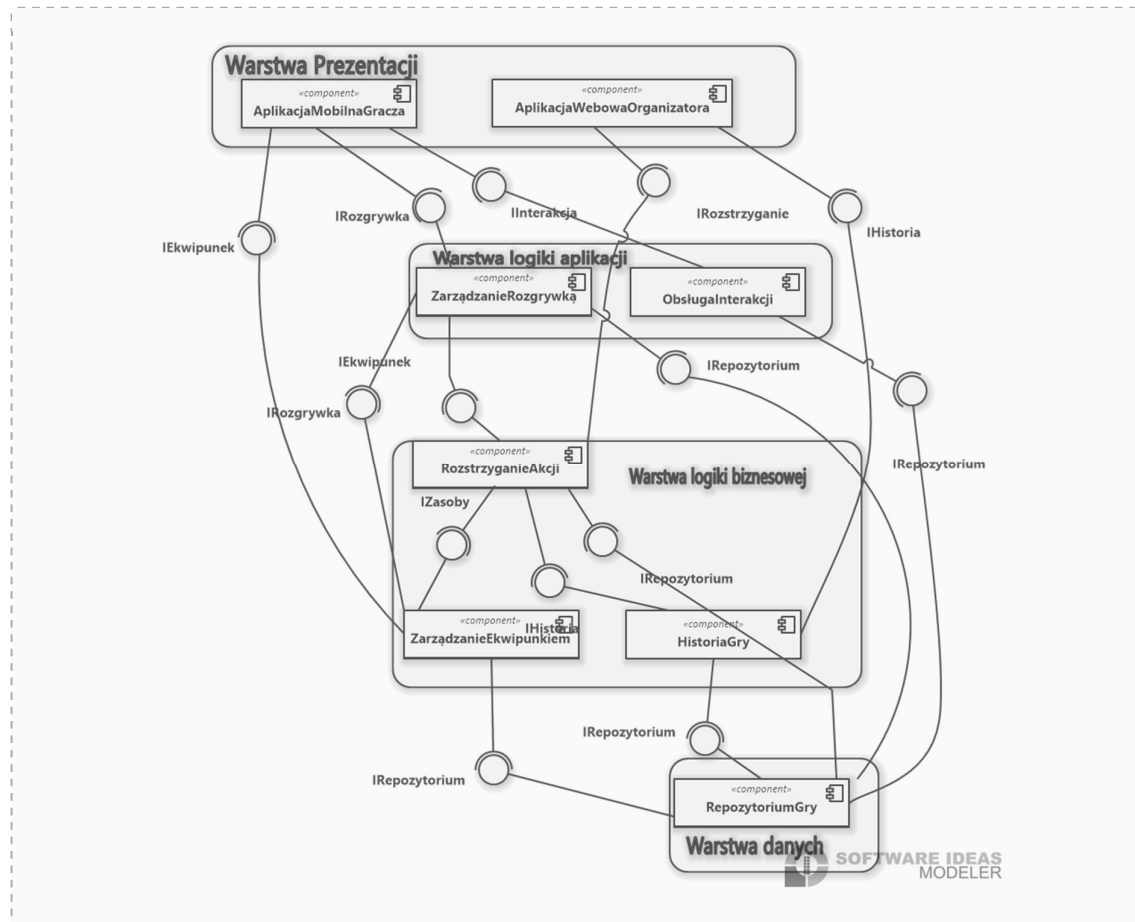
System zaprojektowano w architekturze warstwowej. Warstwa prezentacji odpowiada za interakcję z użytkownikami i obejmuje aplikację mobilną gracza oraz aplikację webową organizatora. Warstwa logiki aplikacji zawiera komponenty sterujące przebiegiem funkcji systemu: ZarządzanieRozgrywką oraz ObsługaInterakcji (odczyt kodów QR i sygnałów czujników). Warstwa logiki biznesowej realizuje reguły dziedzinowe - należą do niej komponenty RozstrzygnięcieAkcji, ZarządzanieEkwipunkiem oraz HistoriaGry; w warstwie tej umieszczono również komponenty integracji z systemami zewnętrznymi (np. AdapterPłatności). Warstwa danych (RepozytoriumGry) przechowuje i udostępnia dane przetwarzane przez system, komunikując się z zewnętrzną bazą danych. Sprzętowe kody QR i czujniki obsługuje komponent BramaUrządzeń.

Taki podział umożliwia realizację przypadków użycia z sekcji 1.1 i 1.2: przykładowo PU001 przebiega przez ObsługaInterakcji → RepozytoriumGry, natomiast PU003 i PU007 angażują RozstrzygnięcieAkcji, ZarządzanieEkwipunkiem oraz HistoriaGry.



Rysunek 6: Diagram komponentów - ogólna struktura systemu (model warstwowy)

Komponenty, interfejsy dostarczane/wymagane oraz zależności między warstwami.

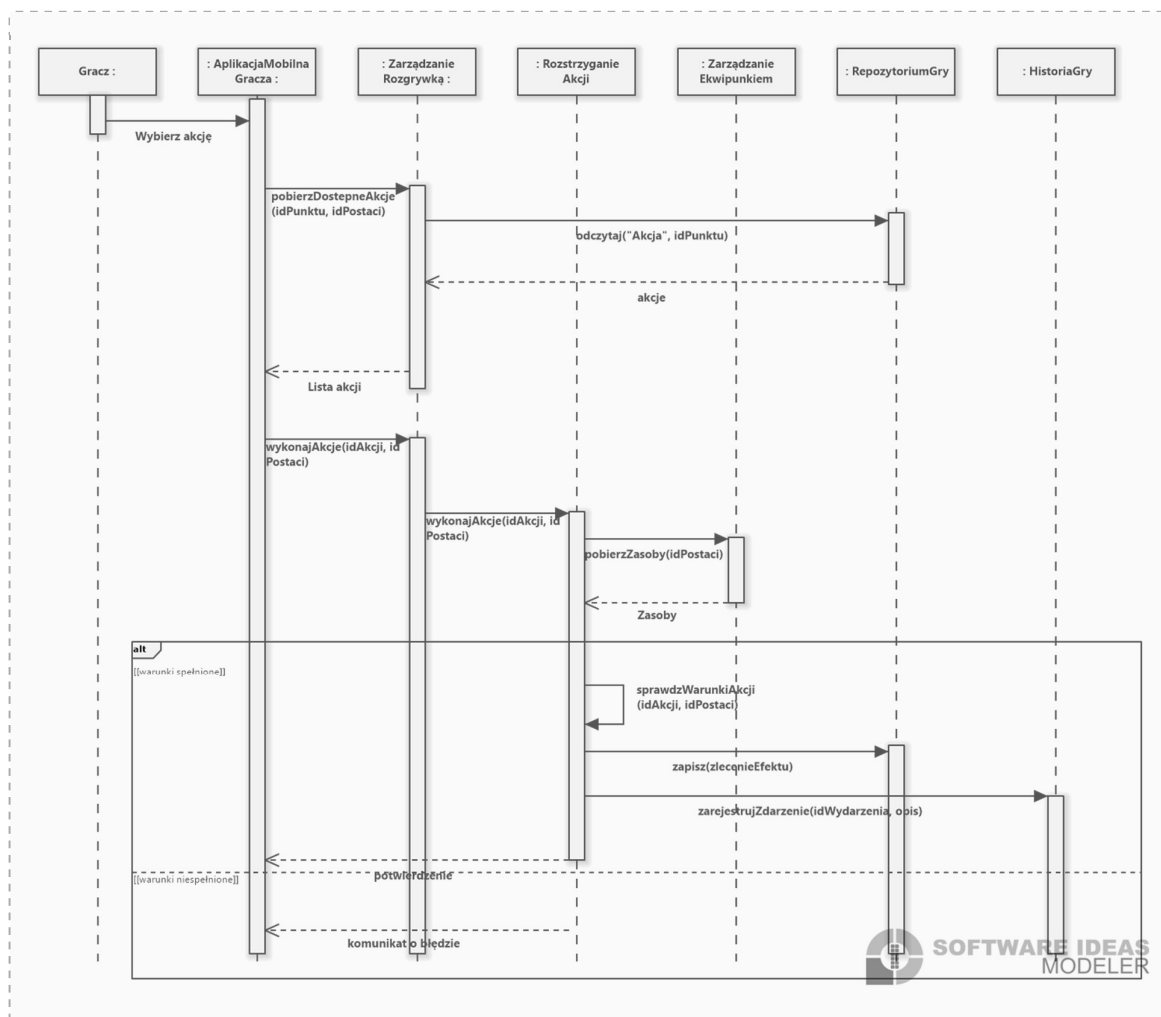


Rysunek 7: Diagram komponentów z interfejsami realizującymi PU sprintu 1

Interfejsy: `IInterakcja`, `IRozgrywka`, `IRozstrzyganie`, `IEkwipunek`, `IZasoby`, `IHistoria`, `IRepozytorium`.

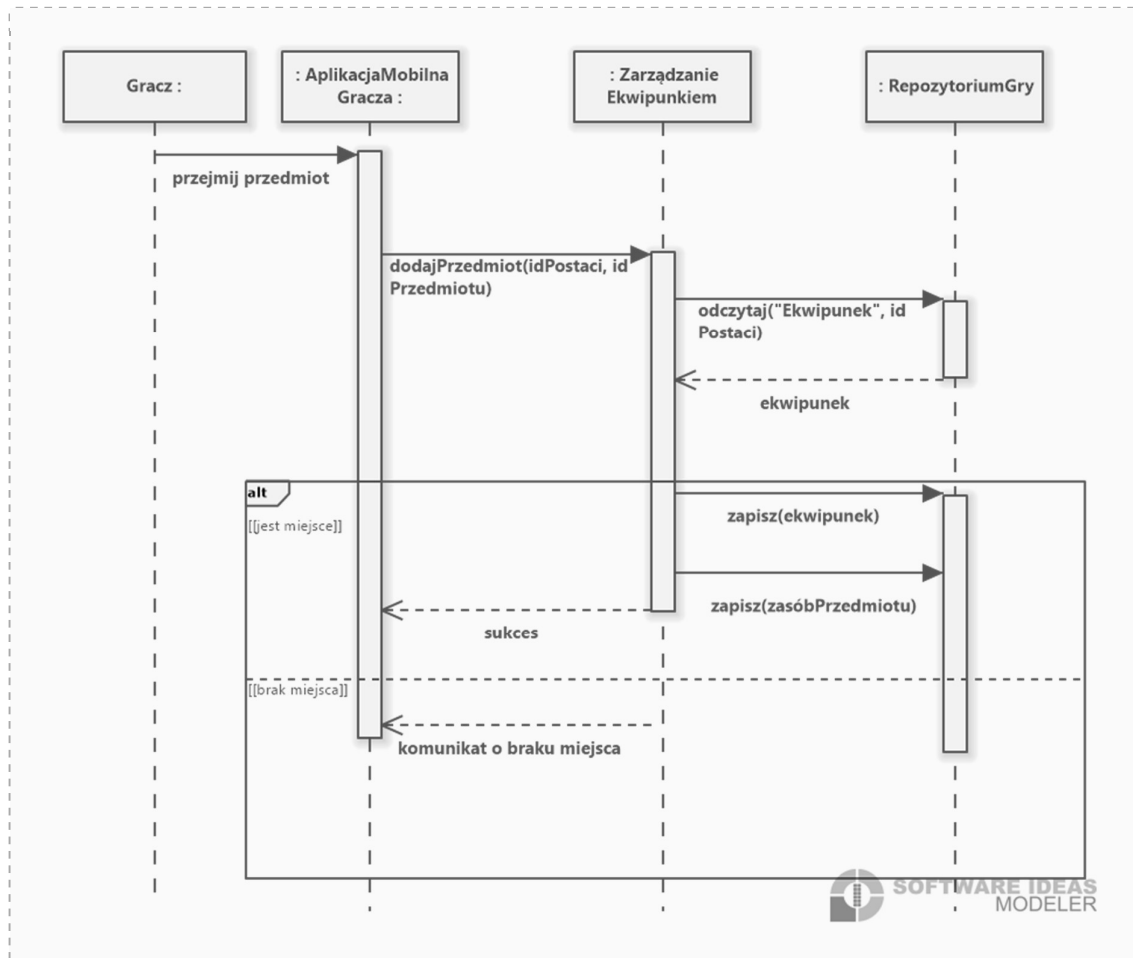
2.2 Projekt dynamiki systemu

Diagramy sekwencji opracowano na poziomie komponentów. Są one spójne ze scenariuszami z sekcji 1.2.



Rysunek 8: Diagram sekwencji - realizacja PU003 (Wykonanie akcji w świecie gry)

Na rysunku 8 przedstawiono realizację PU003. Gracz, poprzez styk aplikacji mobilnej, wywołuje operację `wykonajAkcje()` interfejsu `IRozgrywka` komponentu `ZarządzanieRozgrywką`. Komponent pobiera dostępne akcje z `RepozytoriumGry`, a po wyborze akcji deleguje sprawdzenie warunków i wykonanie do komponentu `RozstrzygnięcieAkcji` (`IRozstrzygnięcie`), który korzysta z `ZarządzanieEkwipunkiem` (`IZasoby`) w celu weryfikacji zasobów. Po spełnieniu warunków reguła akcji jest wykonywana, tworzone jest zlecenie efektu (zapis w `RepozytoriumGry`) oraz wpis w `HistoriaGry`, a użytkownik otrzymuje potwierdzenie. Diagram jest spójny ze scenariuszem PU003 oraz jego diagramem czynności (Rysunek 3). Na diagramie zaznaczono paski aktywacji (czas trwania sterowania) poszczególnych komponentów.



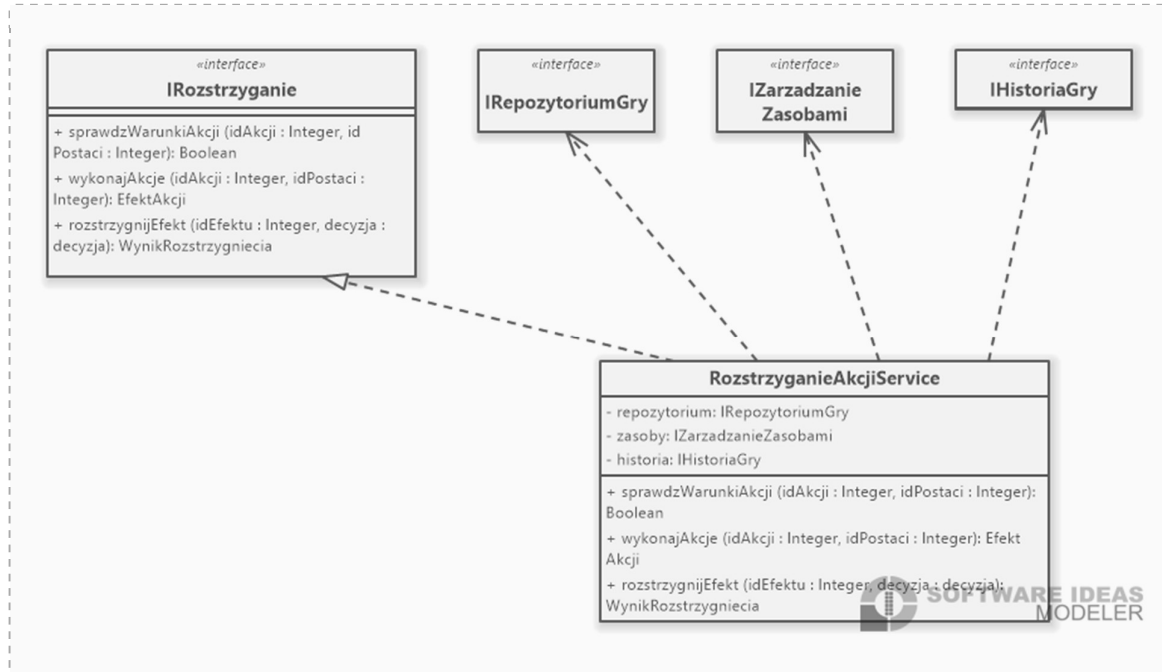
Rysunek 9: Diagram sekwencji - realizacja PU002 (Przejęcie przedmiotu)

Na rysunku 9 przedstawiono realizację PU002. Po wyborze przedmiotu aplikacja mobilna wywołuje `dodajPrzedmiot()` interfejsu `IEkwipunek` komponentu `ZarządzanieEkwipunkiem`, który odczytuje i aktualizuje ekwipunek postaci w `RepozytoriumGry` oraz aktualizuje stan zasobu przedmiotu. Ścieżka alternatywna (brak miejsca) zwraca komunikat bez modyfikacji danych. Na diagramie zaznaczono paski aktywacji komponentów.

3. Projekt podsystemów oraz implementacja

3.1 Realizacja interfejsów

Jako przykład wybrano interfejs `IRozstrzyganie` komponentu `RozstrzyganieAkcji` (sekcja 2.1) oraz realizującą go klasę `RozstrzyganieAkcjiService`. Rysunek 10 przedstawia diagram klas tej realizacji. Poniżej zamieszczono kod interfejsu oraz klasy w języku Java, wygenerowany z modelu UML narzędziem `Software Ideas Modeler` (w identyfikatorach nie stosowano polskich znaków).



Rysunek 10: Diagram klas - realizacja interfejsu IRozstrzyganie

Interfejs IRozstrzyganie oraz realizująca go klasa RozstrzyganieAkcjiService.

Kod interfejsu IRozstrzyganie oraz klasy go implementującej, w języku Java, wygenerowany narzędziem Software Ideas Modeler:

```

/**
 * @author Dawid Bałdyga
 * @version 1.0
 */
public interface IRozstrzyganie {

    boolean sprawdzWarunkiAkcji(int idAkcji, int idPostaci);

    EfektAkcji wykonajAkcje(int idAkcji, int idPostaci);

    WynikRozstrzygnięcia rozstrzygnijEfekt(int idEfektu, Decyzja decyzja);
}
  
```

```

/**
 * @author Dawid Bałdyga
 * @version 1.0
 */
public class RozstrzyganieAkcjiService implements IRozstrzyganie {

    private IRepozytoriumGry repozytorium;
    private IZarzadzanieZasobami zasoby;
    private IHistoriaGry historia;

    public RozstrzyganieAkcjiService() {
    }

    public boolean sprawdzWarunkiAkcji(int idAkcji, int idPostaci) {
        // TODO: sprawdzenie warunkow akcji na podstawie zasobow postaci
        return false;
    }

    public EfektAkcji wykonajAkcje(int idAkcji, int idPostaci) {
    }
}
  
```

```

        // TODO: wykonanie reguły akcji i utworzenie zlecenia efektu
        return null;
    }

    public WynikRozstrzygnięcia rozstrzygnijEfekt(int idEfektu, Decyzja decyzja) {
        // TODO: rozstrzygnięcie efektu, aktualizacja zasobów i historii
        return null;
    }
}

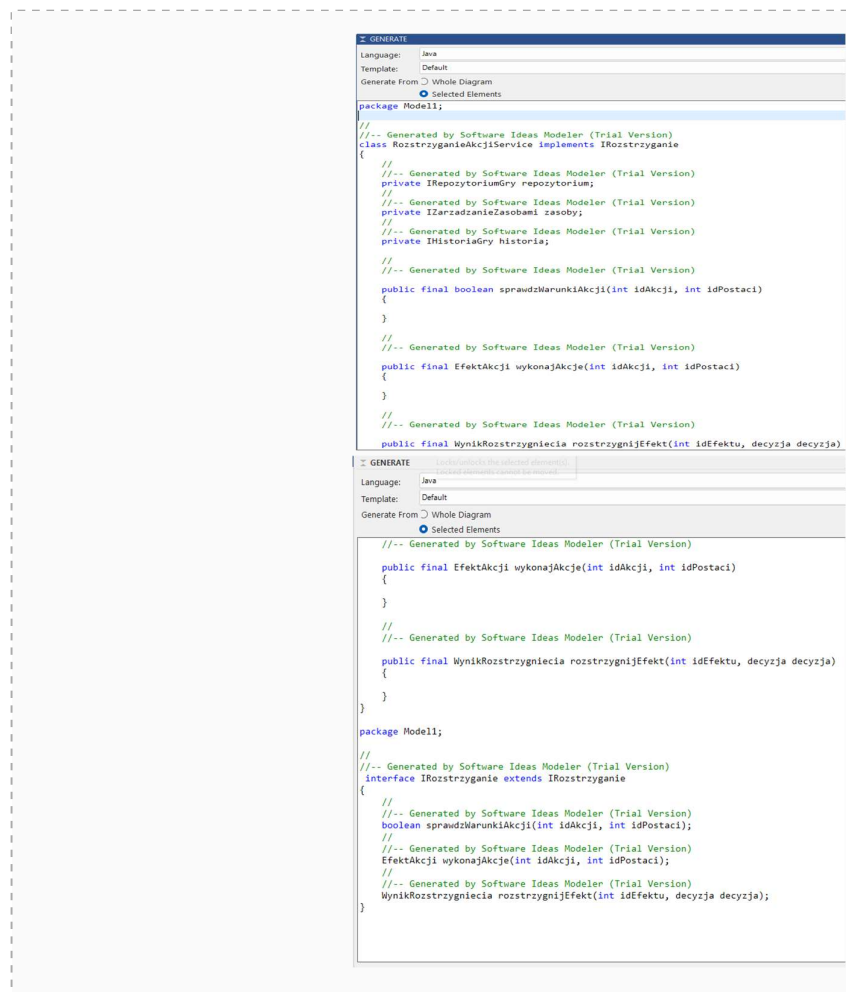
```

3.2 Generacja kodu

Generację kodu w narzędziu Software Ideas Modeler wykonuje się następująco:

- W opcjach projektu lub elementu ustawić docelowy język programowania (np. Java).
- Zadbać o zgodność typów atrybutów i parametrów z wybranym językiem oraz o brak polskich znaków i spacji w nazwach elementów.
- Zaznaczyć klasę/interfejs lub pakiet i wybrać polecenie generacji kodu (Source Code → Generate Code).

Narzędzie tworzy pliki źródłowe odpowiadające elementom modelu; możliwa jest również późniejsza synchronizacja modelu z kodem. Przykład wykonanej samodzielnie generacji przedstawia Rysunek 11.



Rysunek 11: Zrzut ekranu z generacji kodu w narzędziu CASE

Dodatek A: Plan zadań w projekcie

3.3 Rejestr produktowy

Rejestr produktowy zawiera przypadki użycia (nie historie użytkownika). Priorytet odzwierciedla wagę dla klienta, a trudność wykonania oszacowano metodą pokera scrumowego w punktach (skala Fibonacciego). Poniżej przedstawiono przydział do pierwszych trzech sprintów.

Sprint 1

ID	Nazwa przypadku użycia	Priorytet	Trudność (pkt)
PU001	Skanowanie kodu QR	Bardzo wysoki	5
PU002	Przejęcie przedmiotu	Wysoki	3
PU003	Wykonanie akcji w świecie gry	Bardzo wysoki	8
PU004	Podgląd ekwipunku postaci	Średni	2
PU007	Rozstrzygnięcie efektu akcji	Wysoki	8

Sprint 2

ID	Nazwa przypadku użycia	Priorytet	Trudność (pkt)
PU005	Rejestracja obecności w obszarze gry	Wysoki	5
PU006	Zdefiniowanie znaczenia kodu QR / czujnika	Wysoki	5
PU009	Monitorowanie zasobów graczy	Wysoki	5
PU008	Podgląd historii wydarzeń gry	Średni	3

Sprint 3

ID	Nazwa przypadku użycia	Priorytet	Trudność (pkt)
PU010	Zarządzanie terenem gry (pomieszczeniami)	Średni	5
PU011	Definiowanie profili postaci i przedmiotów	Średni	5
PU012	Przegląd przebiegu gry (oś czasu wydarzeń)	Niski	3

3.4 Rejestr sprintu 1

Okres trwania sprintu: 02.03.2026 – 13.03.2026 (2 tygodnie).

ID	Zadanie	Dyscyplina	Osoba odpowiedzialna	Krótki opis	Harmonogram
Z1	Doprecyzowanie scenariuszy PU sprintu 1	Wymagania	Anna Kowalska	Uszczegółowienie scenariuszy RSL-bis i kryteriów akceptacji.	02–04.03
Z2	Projekt modelu komponentów	Projektowanie	Piotr Nowak	Diagram komponentów i interfejsów realizujących PU sprintu.	03–05.03
Z3	Projekt diagramów sekwencji (PU003, PU002)	Projektowanie	Piotr Nowak	Diagramy sekwencji na poziomie komponentów.	05–06.03
Z4	Implementacja ObsługaInterakcji	Implementacja	Marek Wiśniewski	Odczyt i walidacja kodu QR, mapowanie na obiekt gry (PU001).	06–09.03

Z5	Implementacja ZarządzanieEkwipunkiem	Implementacja	Marek Wiśniewski	Przejęcie przedmiotu i podgląd ekwipunku (PU002, PU004).	09–11.03
Z6	Implementacja RozstrzyganieAkcji	Implementacja	Marek Wiśniewski	Reguły akcji, zlecenie i rozstrzyganie efektów (PU003, PU007).	09–12.03
Z7	Testy jednostkowe RozstrzyganieAkcji	Testowanie	Katarzyna Lewandowska	Testy reguł akcji i rozstrzygania efektów.	11–12.03
Z8	Testy akceptacyjne PU sprintu 1	Testowanie	Katarzyna Lewandowska	Testy wg scenariuszy PU001–PU004, PU007.	12–13.03
Z9	Przegląd i retrospektywa sprintu	Zarządzanie	Dawid Bałdyga	Demo przyrostu, retrospektywa, planowanie kolejnego sprintu.	13.03

Zgodnie z definicją sprintu, iteracja kończy się działającym, potencjalnie dostarczalnym przyrostem produktu - tutaj działającą pętlą rozgrywki (skan kodu QR → przejęcie przedmiotu / wykonanie akcji → rozstrzygnięcie efektu → podgląd ekwipunku), zaprezentowaną na przeglądzie sprintu.